# Non-Blocking Audio Architecture for ESP32 + NRF24

## Goal

Integrate real-time audio streaming into a larger ESP32 application without using FreeRTOS tasks.

The existing project already uses hardware timers and other time-sensitive systems, so audio should operate using a cooperative, non-blocking architecture that can be called from the main loop.

Instead of multiple RTOS tasks, the system will be organized into update functions and circular buffers.

---

# Design Philosophy

The original implementation used:

- micTask()
- nrfTask()
- dacTask()

These tasks continuously ran in parallel.

For integration into a larger codebase, this approach can become difficult to manage because:

- Multiple execution contexts exist
- Timing becomes harder to reason about
- Debugging becomes more difficult
- Interactions with other subsystems become less predictable

Instead, the recommended architecture is:

```
loop()
├── updateAudioCapture()
├── updateRadioTx()
├── updateRadioRx()
├── updateAudioPlayback()
├── updateUI()
├── updateGame()
├── updateTimers()
└── updateInputs()
```

Each subsystem executes quickly and returns immediately.

No function should block.

---

# System Overview

## Transmitter

Audio flow:

```
INMP441
    ↓
I2S DMA
    ↓
updateAudioCapture()
    ↓
Audio Ring Buffer
    ↓
updateRadioTx()
    ↓
NRF24
```

---

## Receiver

Audio flow:

```
NRF24
    ↓
updateRadioRx()
    ↓
Audio Ring Buffer
    ↓
Timer Driven DAC Output
    ↓
GPIO25 DAC
    ↓
LM386
    ↓
Speaker
```

---

# Circular Buffer Design

The ring buffer becomes the key component.

It decouples:

- Audio acquisition
- Radio transmission
- Radio reception
- Audio playback

Without requiring separate tasks.

Example:

```
#define AUDIO_BUFFER_SIZE 2048

uint8_t audioBuffer[AUDIO_BUFFER_SIZE];

volatile uint16_t head = 0;
volatile uint16_t tail = 0;
```

Functions:

```
bool audioAvailable();
bool audioSpaceAvailable();

void pushAudio(uint8_t sample);
uint8_t popAudio();
```

The ring buffer allows timing differences between subsystems without blocking execution.

---

# Transmitter Design

## updateAudioCapture()

Purpose:

- Read microphone samples from I2S
- Remove DC offset
- Apply gain
- Convert to 8-bit PCM

- Push samples into transmit buffer

Called continuously from loop().

Example:

```
void updateAudioCapture()
{
    size_t bytesRead;

    i2s_read(
        I2S_NUM_0,
        i2sBuf,
        sizeof(i2sBuf),
        &bytesRead,
        0
    );

    if(bytesRead == 0)
        return;

    processSamples();
}
```

Important:

The timeout argument is:

```
0
```

instead of:

```
portMAX_DELAY
```

This guarantees the function never blocks.

---

## Audio Processing

For each microphone sample:

### 1. Extract sample

```
sample24 = i2sBuf[i] >> 8;
```

### 2. Remove DC offset

```
dc_offset =
    (dc_offset * 63 + sample24) / 64;
```

```
centered =
    sample24 - dc_offset;
```

### 3. Apply gain

```
scaled =
    (centered * GAIN) >> 16;
```

### 4. Convert to DAC-compatible range

```
value = 128 + scaled;
```

### 5. Clamp

```
0 ≤ value ≤ 255
```

### 6. Push to transmit buffer

```
pushAudio(value);
```

---

## updateRadioTx()

Purpose:

Build NRF24 packets whenever enough audio samples are available.

Packet format:

```
Byte 0  : Sequence Number
Byte 1-31 : Audio Samples
```

Called continuously from loop().

Example:

```
void updateRadioTx()
{
    while(audioAvailable())
    {
        packet[idx++] = popAudio();

        if(idx == 32)
        {
            packet[0] = seq++;

            radio.writeFast(packet, 32);

            idx = 1;
        }
    }
}
```

Characteristics:

- Non-blocking
- No delays
- No RTOS task

---

# Receiver Design

## updateRadioRx()

Purpose:

Receive packets and store samples into playback buffer.

Called continuously from loop().

Example:

```
void updateRadioRx()
{
    while(radio.available())
    {
        radio.read(packet,32);

        for(int i=1;i<32;i++)
        {
            pushAudio(packet[i]);
        }
    }
}
```

Characteristics:

- Non-blocking
- Drains radio FIFO quickly
- Returns immediately when no packets exist

---

# Audio Playback

Audio playback requires precise timing.

The DAC must receive one sample every:

```
1 / 8000
=
125 microseconds
```

Using delays inside loop() is not recommended.

---

# Hardware Timer Driven Playback

Recommended architecture:

```
volatile bool audioTick = false;

void IRAM_ATTR onAudioTimer()
{
```

```
    audioTick = true;
}
```

Timer frequency:

```
8000 Hz
```

Period:

```
125 microseconds
```

---

## updateAudioPlayback()

Called from loop().

```
void updateAudioPlayback()
{
    if(!audioTick)
        return;

    audioTick = false;

    if(ringTail != ringHead)
    {
        dac_output_voltage(
            DAC_CHANNEL_1,
            ring[ringTail]
        );

        ringTail =
            (ringTail + 1)
            & RING_MASK;
    }
    else
    {
        dac_output_voltage(
            DAC_CHANNEL_1,
            128
        );
    }
}
```

Benefits:

- Precise playback rate
- Non-blocking
- Easy integration

---

# Alternative: DAC Inside Timer ISR

An even simpler architecture is possible.

The timer ISR can directly output samples.

Example:

```
void IRAM_ATTR onAudioTimer()
{
    if(ringTail != ringHead)
    {
        dac_output_voltage(
            DAC_CHANNEL_1,
            ring[ringTail]
        );

        ringTail =
            (ringTail + 1)
            & RING_MASK;
    }
    else
    {
        dac_output_voltage(
            DAC_CHANNEL_1,
            128
        );
    }
}
```

In this design:

- Main loop only fills buffers
- Playback timing is completely hardware-driven

This is often the preferred solution for simple voice communication systems.

---

# Main Loop Structure

Final architecture:

```
void loop()
{
    updateAudioCapture();

    updateRadioTx();

    updateRadioRx();

    updateAudioPlayback();

    updateUI();

    updateButtons();

    updateGameLogic();

    updateTimers();
}
```

Characteristics:

- No delays
- No blocking reads
- No RTOS tasks
- Deterministic execution
- Easy integration with existing systems

---

# Important Timing Notes

Audio playback must remain:

```
8000 Hz
```

or

```
16000 Hz
```

depending on the chosen sample rate.

A timer running at:

```
1000 Hz
```

or

```
100 Hz
```

cannot directly drive audio playback.

Audio should have:

- Its own hardware timer, or
- A timer already configured at the required audio sample rate

---

# Advantages of the Non-Blocking Architecture

Compared to the original RTOS implementation:

- Easier integration into existing projects
- Fewer execution contexts
- Deterministic behavior
- Simpler debugging
- Lower scheduling overhead
- Better compatibility with custom timing systems
- No task watchdog issues
- No FreeRTOS dependency for audio processing

---

# Summary

The recommended integration strategy is to replace the original audio tasks with a cooperative update-based architecture.

Audio capture, radio transmission, radio reception, and playback all operate through shared ring buffers and non-blocking update functions.

Precise playback timing is maintained using a dedicated hardware timer running at the audio sample rate.

This preserves the original functionality while making the audio subsystem significantly easier to integrate into larger ESP32 applications.